Module IV: Shell Scripting

- Introduction to Shell Scripting: Overview of shell scripting and its purpose, Types of shells (e.g., Bash, Zsh, Ksh, Csh), Differences between command line and shell scripts.
- Writing First Shell Script: Script structure and syntax,Creating and executing a simple shell script, Using the shebang (#/!) for specifying the interpreter.
- User management. Variables and Data Types: Declaring and using variables, Variable scope and export, Special variables. Control Structures: Conditional statements (if, else, elif,case), Looping constructs (for, while, until), Using break and continue in loops. Functions: Defining and calling functions, Passing arguments to functions, Returning values from functions.
- Input and Output: Reading user input with read, Redirecting input and output, using pipes to connect commands.

- A shell script is a computer program designed to be run by the Unix/Linux shell which could be one of the following:

- The Bourne Shell
- The C Shell
- The Korn Shell
- The GNU Bourne-Again Shell

- A shell is a command-line interpreter and typical operations performed by shell scripts include file manipulation, program execution, and printing text.

- Shell Prompt
- The prompt, $, which is called the command prompt, is issued by the shell.
- While the prompt is displayed, we can type a command.

- Bourne shell – using a Bourne-type shell, the $ character is the default prompt.

- C shell – a C-type shell, the % character is the default prompt.

- Creating and running a shell script

- A shell script is a file that contains one or more commands.
- Shell scripts provide an easy way to carry out tedious commands, large or complicated sequences of commands, and routine tasks.
- When the name of a shell script file is entered , the system executes the command sequence contained by the file.
- Script can contain both operating system commands and shell built-in commands.

- create a new file with the .sh extension

- #!/bin/bash
- echo "Hello World“

- $ chmod u+x myfirstscript.sh
- $ ./myfirstscript.sh
- Hello World

- #!/bin/bash which signifies that command interpreter should be the bash (Bourne Again Shell) shell.
- Without first line also the script will be run.
- If the line is included :
- It defines a shell script that it starts its shell instance(often called a “subshell”), runs a series of commands within that subshell so it is completely independent or isolated from the user shell which is logged in. This is very useful and sometimes needed to have this separation so current environment is not affected — having the #!/bin/bash allows us to do that.

- Shell Scripts
- The basic concept of a shell script is a list of commands, which are listed in the order of execution. A good shell script can have comments, preceded by # sign, describing the steps.
- Variable Names
- The name of a variable can contain only letters (a to z or A to Z), numbers (0 to 9) or the underscore character (_).
- Defining Variables
- variable_name=variable_value
- No1=12
- No2=3
- To access the value stored in a variable, prefix its name with the dollar sign ($)

- The shell is, programming language, complete with variables, control structures, and so forth. it is still just a list of commands executed sequentially.
- the read command takes the input from the keyboard and assigns it as the value of the variable name and finally prints it on STDOUT.

-
```
#!/bin/sh
echo "What is your name?"
read name
echo "Hello, $name"
```

- Shell Decision Making:
- Shell supports conditional statements which are used to perform different actions based on different conditions.
- If else statements are useful decision-making statements which can be used to select an option from a given set of options.

- if...fi statement
- if...else...fi statement
- if...elif...else...fi statement

```
If test-command
then
commands
Fi
```

```
#! Bin/sh
A=10
B=20
 If [$a -eq $b ] then
echo “ Both are equal”
fi
```

- num=15
- if [$num -gt 10]; then
- echo "Number is greater than 10"
- Fi

- `-eq  -ne  -gt -lt -ge -le`

```
if [ condition ]; then
   # commands if true
elif [ another_condition ]; then
   # commands if second condition is true
else
   # commands if all conditions are false
fi
```

```
case $variable in
    pattern1)
        # commands
        ;;
    pattern2|pattern3)
        # commands for multiple matches
        ;;
    *)
        # default case (catch-all)
        ;;
esac
```

- Shell Loop Types
- A loop is a powerful programming tool that enables to execute a set of commands repeatedly.
- The while loop
- The for loop

```
a=0
while [ "$a" -lt 10 ]    # this is loop1
  do
echo -n "$a “
 a=` expr $a + 1 `
done
```

- Functions in Script:
- To declare a function, use the following syntax –

```
function_name ()
 {
   list of commands
  }
```

Exampe :

```
#!/bin/sh
# Define your function here
  Hello () {
    echo "Hello World"
   }

 # Invoke  function
 Hello
```

- Pass Parameters to a Function
- Define a function that will accept parameters while calling the function.
- These parameters would be represented by $1, $2

- echo "Hello World $1 $2“

- Hello PICT College

- Returning a value from function
- echo "Hello World $1 $2"
- return 10

Capturing the return value

- ret=$?

- Using the return Value.
- echo "Return value is $ret"

- Understanding the /etc/passwd file
- User account information is stored in the /etc/passwd file. This information includes the account name, home directory location, and default shell, among other values. Each field is separated by a : character.
- an example of the /etc/passwd fields:
- username:password:UID:GID:comment:home:shell
- Create, modify, and delete user accounts
- The commands to manage user accounts
- useradd
- usermod
- userdel

- Understand the /etc/group file
- Similar to the /etc/passwd file above, the /etc/group file contains group account information. This information can be essential for troubleshooting, security audits, and ensuring users can access the resources they need.

- Each field of the file is significant as a sysadmin.

- The fields in the /etc/group file are:
- groupname:password:GID:group members
- Example of the **editors** group with two members:
- editors:x:2002:damon,tyler

- Create, modify, and delete groups
- Like the user account commands described above, the group management commands are very intuitive and provide a lot of flexibility. There is an easy-to-remember command for each function you might need to carry out for a group:

- groupadd
- groupmod
- groupdel

- Manage group membership
- Adding users to a group simplifies permissions management. Many people find the process a little unintuitive: Adding a user to a group modifies the user, not the group. Therefore, the necessary command is the usermod command.

- Some commands to display group information:

- usermod: Update group membership
- id: Display a list of groups the user is a member of
- cat /etc/group: Show a list of existing groups, with membership displayed in the last field

- useradd command to create the user with the -m flag, which ensures the user's home directory is created.
- The -s /bin/bash flag sets the user's default shell to Bash.

- To set the user’s password, chpasswd command is used in combination with an echo command to provide the username and password in the required format.

- Run the script :
- sudo ./create_users.sh

```
#!/bin/bash
# Function to create a new user
create_user() {
  username=$1
  password=$2
  # Create the user with the specified username
  sudo useradd -m -s /bin/bash $username
  # Set the user's password
  echo "$username:$password" | sudo chpasswd
  echo "User '$username' has been created with the password '$password’”
}
# Create two users
create_user user1 password1
create_user user2 password2
```